Building and Analyzing Monolingual Transformer LMs in Two Indian Languages

Consolidated report, Phases 1–3 · CL3410, Monsoon 2026

Sourav Nath (2025201054) · **Model H** = Hindi (Devanagari), **Model L** = Assamese (Bengali–Assamese)

I built two decoder-only Transformers of **24,086,016** parameters each from primitive PyTorch layers, pretrained them on separate monolingual corpora, and finetuned each on synthetic reasoning data in its own language. The two models share no data, tokenizer, vocabulary or weights.

One design decision shapes everything that follows: the two pretraining token budgets differ by only 0.2%, so data quantity is held constant and any gap between the models has to come from tokenizer fertility or corpus composition instead. Model H reaches **0.5595** bits-per-byte against Model L's **0.6238**. On reasoning, both start at **0.0000** and finetune to **0.8460** and **0.7555**. The error analysis then complicates that headline: both models select the correct value at essentially the same rate (97.50% and 98.00%), so the gap between them is not about comparing numbers but about keeping track of which entity owns which value.

Corpora	https://drive.google.com/drive/folders/1jLpgQBmY3gGCWfqxP_Kz_4MQvQHxoGJS?usp=sharing
Pretrained	https://drive.google.com/drive/folders/1bIOG67ZM5iARPTvk3FGR51DxqIIxlLLn?usp=sharing
Finetuned	https://drive.google.com/drive/folders/1W9mShiOj3JrejAjZgH6QvGRv99J4ksfK?usp=sharing

All three are shared as *anyone with the link → viewer*; no access request is needed.

1 Phase 1: corpora and tokenizers

Everything in the manual tier was collected with my own code. Wikipedia and Wikisource come from the official XML dumps through a wikitext extractor I wrote, rather than by scraping the live site. News came from sitemap scrapers for `livehindustan`, `prabhatkhabar` and `jagran` in Hindi, and `asomiyapratidin` and `xahitya.org` in Assamese. Finding those two Assamese sources took probing more than twenty candidate domains; most had no usable sitemap or too little volume to matter. Books came from archive.org via my own OCR-text collector. The downloaded tier is Sangraha, IndicCorpV2, MADLAD-400, Glot500 and two Assamese corpora from HuggingFace. I never used Sangraha's *synthetic* config. Downloaded rows go through the same language-identification stage as manual text, because a spot check found rows in Sangraha that were not in the language they claimed to be.

Cleaning runs in six stages, and the order matters. Unicode NFC normalisation comes first (this is also where native digits are folded to ASCII, so that `2024` and its Devanagari spelling do not occupy two vocabulary entries), then language identification, then boilerplate and quality heuristics, then non-language word removal, then machine-translation exclusion for Wikipedia, then deduplication. I originally ran the non-language cascade before boilerplate removal and lost 64% of Hindi news documents, because datelines like `IST` and `Aug` were being counted as non-language words rather than as the code-mixing the stage is meant to catch. Swapping the two stages fixed it without loosening any threshold.

Thresholds differ between the languages on purpose. Hindi has a surplus, so it filters for precision and drops 14.4% of news words, mostly datelines. Assamese is genuinely scarce, so it filters gently (0.96%) and repairs documents by stripping the offending portion rather than discarding them. I tuned both by reading actual rejected documents in each language.

Deduplication is three passes: exact SHA-256, then paragraph-level (a paragraph appearing in five or more documents is dropped everywhere, which catches navigation blocks), then near-duplicate MinHash with LSH, using k=100 hashes in b=20 bands of r=5 over 3-word shingles, giving a similarity threshold near 0.55. Splits are 98/1/1, assigned by a deterministic hash of the document id so that re-running the pipeline cannot move a document from train into test. A decontamination pass then checks train against the held-out splits for near-duplicates.

Table 1: Corpus and tokenizer statistics, measured with each language's own final tokenizer.

Corpus	Hindi	Assamese	Tokenizer (held-out)	Hindi	Assamese
train documents	989,016	824,317	Vocabulary (BPE)	16,000	16,000
train words	453,479,963	373,902,951	**Fertility (tok/word)**	**1.3067**	**1.6019**
train tokens	**589,952,668**	**591,255,819**	Characters per token	3.9288	4.0194
manual share	**20.80%**	**23.88%**	Byte-fallback rate	0.4295%	0.6502%
validation tokens	6,980,595	8,350,610	True UNK (no-fallback)	0.1187%	0.1855%
test tokens	6,800,389	8,158,501	Distinct tokens in val	15,861	15,880

Choosing the vocabulary size took three attempts. A fertility sweep across 8k to 64k minimises at 64k, but minimising fertility alone is the wrong objective, because the embedding layer counts against Phase 2's 25M-parameter budget and 32k at d=384 already consumes 49% of it even with tied embeddings. I moved to 8k, which is safe but fragments words badly, and finally settled on 16k with tied embeddings. That keeps embeddings at 25.5% of the budget while recovering most of the fertility lost at 8k (Hindi 1.4318 to 1.3135 on the sweep's own measurement). Both corpora clear the 500M-token target and the 20% manual floor, and Assamese ends up with the higher manual share of the two, which took six rounds of expansion to achieve.

2 Phase 2: architecture and pretraining

The model is a GPT-style decoder-only Transformer written from `nn.Linear`, `nn.Embedding`, `nn.LayerNorm` and `nn.Dropout`. Multi-head attention, the learned positional embeddings and the causal mask are all written out explicitly in `common/modeling/model.py`.

Table 2: Architecture (identical for both models) and the exact parameter split.

Hyperparameter	Value	Hyperparameter	Value	Parameter group	Count
Layers N	10	Context T	512	Token embedding	6,144,000
Heads h	6	Dropout	0.1	Positional embedding	196,608
d_{model}	384	Vocabulary	16,000	Transformer blocks	17,744,640
d_{ff}	1,536	Embeddings	tied	Final LayerNorm	768
$d_k = d/h$	64	Norm	pre-norm	**Total trainable**	**24,086,016**

I used learned absolute positional embeddings. With a fixed 512-token context and hundreds of millions of training tokens there is plenty of data to learn position representations directly. The cost is that a learned table has exactly `block_size` rows, so the model cannot process a longer sequence than it trained on; the forward pass raises an error rather than silently truncating. Sinusoidal encodings and RoPE both extrapolate and learned ones do not, but since every evaluation here runs at the training context length, that limitation is never exercised.

Normalisation is pre-norm, applied to each sublayer's input rather than its output. This leaves an unnormalised residual path running from the embedding to the final norm, so gradients reach the early layers without passing through a LayerNorm at every step. Post-norm stacks usually need more careful warmup and initialisation to stay stable at depth, which is a poor trade when a ten-layer stack gets exactly one training run on a limited GPU budget. Tying the input and output embeddings saves $16000 \times 384 = 6{,}144{,}000$ parameters, putting embeddings at 25.5% of the model rather than 40.5%, and that saving is the specific reason a 16k vocabulary was affordable at all. I chose depth over width, ten layers at d=384 rather than six at 512, on the reasoning that chained premises benefit from more sequential composition steps. Phase 3's perfect transitive scores are at least consistent with that.

Attention is computed per head as $\text{softmax}\big(QK^\top/\sqrt{d_k} + M\big)\,V$. The $1/\sqrt{d_k}$ term is needed because $QK^\top$ sums d_k roughly unit-variance products, so the logits grow with head width and push the softmax toward a one-hot distribution where the gradient nearly vanishes. At d_k=64 they would be about eight times larger without it.

M is an additive mask holding $-\infty$ on future positions, applied *before* the softmax so those weights

come out at exactly zero while the remaining row still sums to one. Zeroing the probabilities afterwards would leave rows that no longer normalise. The mask is a registered buffer rather than a parameter: fixed geometry that must not receive gradients but should still move with `.to(device)`.

I verify this empirically in `verify_model.py` — changing token $t{+}1$ leaves the logits at position t bit-identical. That is one of six checks, and all six pass.

Training used AdamW ($\beta = 0.9, 0.95$, weight decay 0.1 applied to matrices but not to biases or LayerNorm gains, gradient clipping at 1.0), a peak learning rate of $3{\times}10^{-4}$ with 500 warmup steps decaying by cosine to $3{\times}10^{-5}$, in fp16 with a GradScaler. Each model saw 6,000 steps of 98,304 tokens, so **589,824,000 tokens**, roughly one epoch, taking 290 and 263 minutes on a Kaggle T4. Checkpoints are written atomically via a `.tmp` file and `os.replace`, and carry weights, optimizer state, the step (the schedule is a pure function of it, so nothing else is needed to resume) and both configs. Kaggle caps a session at twelve hours and can evict sooner, so the resume path was exercised for real rather than merely implemented.

One compute problem is worth recording because it nearly cost the project a run. My first attempt selected `bfloat16`, since `is_bf16_supported()` returns true on a T4. It does return true, but a T4 is Turing (`sm_75`) and has fp16 tensor cores only, so bf16 runs emulated. That measured 8.52 s/step, which projects to 14.2 hours and breaks the session limit. Gating the dtype on compute capability instead of on the support flag gave 2.64 s/step, a 3.2× speedup, and is what made the runs fit.

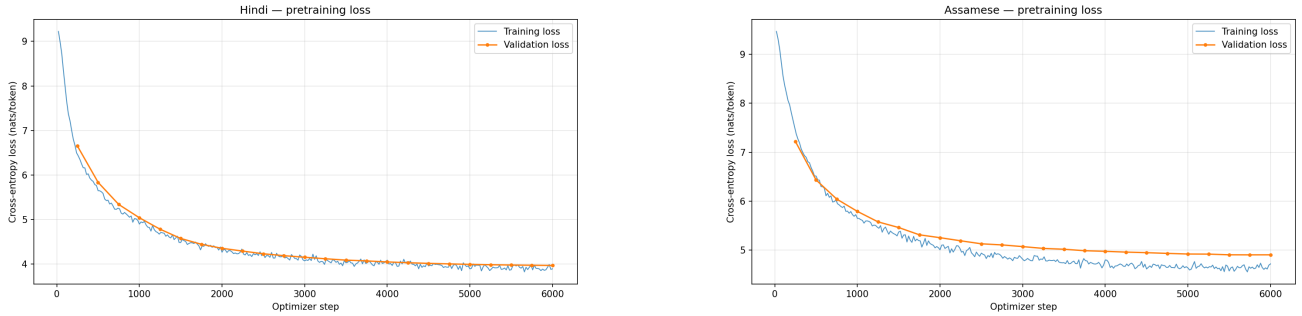


Figure 1: Pretraining loss for Model H (left) and Model L (right). Neither run diverges or spikes, and validation tracks training throughout, which is expected when a single epoch over a 590M-token corpus leaves almost no room to overfit. Model L settles at a higher loss (final training 4.718 against 3.896), the same gap that appears later as bits-per-byte.

3 Phase 2: language-modelling results

Table 3: Held-out test split. Lower is better for the intrinsic metrics.

Intrinsic	H	L	L/H	Generation	BLEU-4	chrF	R-L	D-1	D-2	Rep-4
Cross-entropy	3.8864	4.5556	1.17×	H greedy	1.76	12.16	.102	.066	.128	.824
Perplexity	**48.73**	**95.16**	**1.95**×	H τ=0.5	2.68	17.41	.132	.142	.353	.444
Bits-per-byte	**0.5595**	**0.6238**	**1.11**×	H τ=1.0	1.58	21.40	.116	.381	.816	.072
Tokens scored	6,811,136	8,169,472		H τ=1.5	0.12	19.31	.058	.588	.952	.034
Bytes scored	68,252,613	86,077,062		L greedy	2.47	11.24	.059	.079	.117	.856
Bytes per token	10.021	10.536		L τ=0.5	2.89	14.98	.081	.153	.299	.563
				L τ=1.0	2.97	22.27	.074	.475	.881	.066
				L τ=1.5	0.11	20.73	.032	.655	.983	.013

Perplexity and bits-per-byte disagree about the size of the gap, and the disagreement is instructive. Perplexity puts Model L 1.95× behind, bits-per-byte only 1.11×, from the same predictions on the same text. Perplexity is measured per token, and a token is not a comparable unit across two different tokenizers. Assamese segments the same quantity of language into more, shorter pieces, so Model L makes more predictions to cover the same text (8.17M tokens for 86.1M bytes, against 6.81M for 68.3M),

and since PPL $= e^{\mathrm{CE}}$, that per-token difference gets exponentiated. Bits-per-byte renormalises onto UTF-8 bytes, which both languages share, via BPB $= \mathrm{CE} \cdot \text{tokens}/(\ln 2 \cdot \text{bytes})$. Reporting 1.95× as the gap would overstate it by roughly a factor of 1.75.

The generation metrics deserve more scepticism than they usually get. BLEU-4 requires exact 4-gram overlap with a single reference, but continuing a held-out prefix admits many fluent and correct continuations of which only one is in the reference. Model L scores higher BLEU-4 than Model H at three of the four decoding settings while being clearly the weaker model on bits-per-byte, which tells you the metric is failing rather than that Model L is winning. chrF scores character n-grams and suits morphologically rich Indic scripts better, and ROUGE-L is the steadiest of the three here. The diversity diagnostics are more informative and behave exactly as temperature theory predicts: greedy decoding degenerates with 82–86% of 4-grams repeating, τ=1.0 sits at the useful point, and τ=1.5 buys near-maximal Distinct-2 of 0.95–0.98 while BLEU-4 collapses by a factor of twenty.

Generated samples, τ=1.0, continuing a held-out prefix (96 new tokens, truncated here).

H साइटैटॉजी, आर्टिफिसोपोटोलॉजी, मैकेनिकल, रसायनशास्त्र, वैज्ञानिक अध्ययन और बीमारियों और कमजोर दक्षिण अमेरिका में...

Grammatical connectives and plausible Devanagari morphology, but the content drifts and some terms are invented. Local fluency without discourse coherence is what a 24M model at this budget should be expected to produce.

L নাম হৈছে পোটাৰ আণ্ডাৰ খান আৰু তেওঁৰ পত্নী ক্ৰমে উদিত দাস, মুকেশ বিবি ৰহমান। ছবিখনৰ নামটো হৈছে...

Continues the film-news register of its prompt and keeps sentence structure intact, though the named entities are confabulated.

4 Attention analysis

I computed per-head normalised entropy (1 means uniform over the allowed positions) and mean attention distance for both models on a real 48-token held-out passage, and then again for the pretrained and finetuned checkpoints on a fixed transitive prompt so that the two conditions are comparable cell by cell.

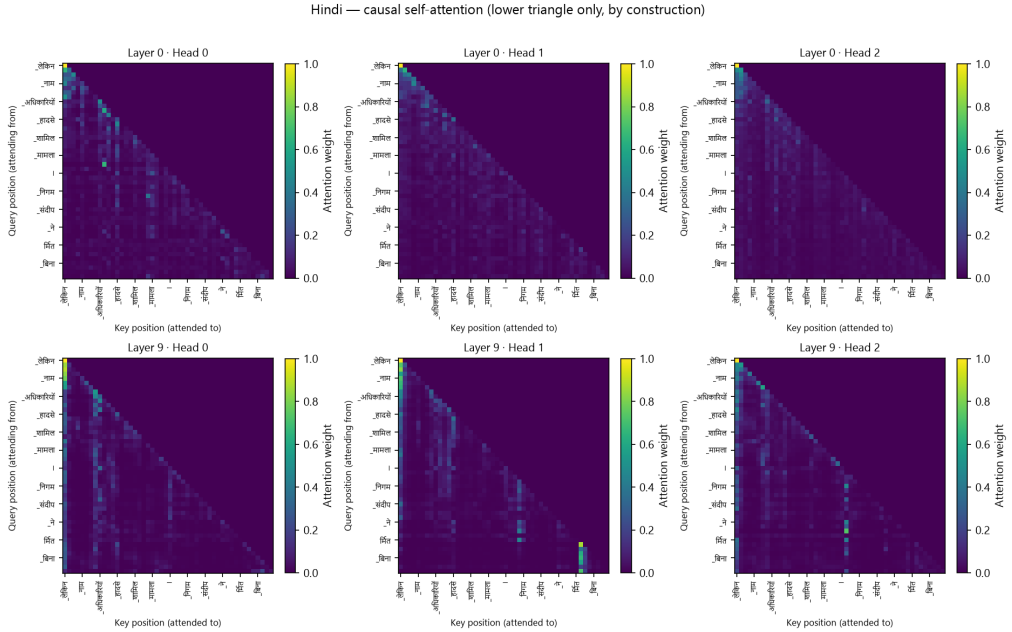
In the pretrained models the early layers are diffuse and short-range, with entropy around 0.93, which fits their job of assembling subword pieces into words. The middle layers are the sharpest in the stack, with Hindi layer 5 at 0.505 entropy. The late layers have the longest mean distance (Hindi layer 9 at 9.82 tokens) and show the vertical striping visible in Figure 2, which indicates content-based rather than positional heads. Model L's heads run slightly longer-range throughout, consistent with a less well-fitted model spreading its attention wider.

Table 4: Mean attention distance and entropy, pretrained to finetuned, on the same prompt.

	Model H			Model L		
	pre	post	Δ	pre	post	Δ
Distance, early L0–L4	6.51	6.72	+0.21	6.38	6.62	+0.24
Distance, late L5–L9	9.50	7.73	**−1.76**	9.86	6.66	**−3.20**
Entropy, late L5–L9	0.634	0.687	+0.053	0.636	0.674	+0.037
Heads becoming shorter-range		38 / 60			**42 / 60**	

Finetuning made late-layer attention considerably more local and left the early layers essentially untouched. Late-layer mean distance fell by 1.76 tokens in Model H and 3.20 in Model L, which on a 34–36 token prompt are contractions of 19% and 32%, while the early layers moved by only 0.2 tokens. That asymmetry has a straightforward reading. Early layers do local, largely positional work that reasoning finetuning gives no reason to change, whereas the late layers had been surveying a 512-token context in order to predict ordinary text. In a reasoning prompt everything the model needs is nearby and specific: the premise clause, the compared value, the entity that owns it. So the late layers stop surveying and start fixating. Head specialisation sharpens rather than reorganises, since no head flips from local to long-range and the heads that were already longest-range contract the most (Model H layer 8 head 2 from 10.61 to 6.87; Model L layer 5 head 2 from 13.02 to 7.51). Model L moves nearly twice as far, which fits finetuning being a stronger perturbation to the weaker model.

Pretrained, on a held-out Hindi passage



Finetuned, on a transitive reasoning prompt

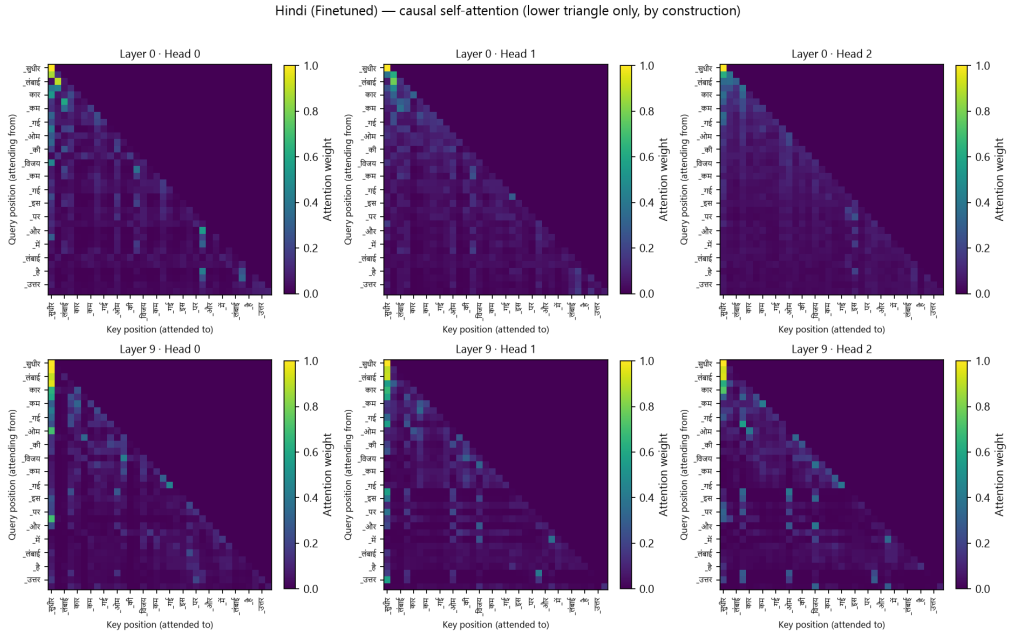


Figure 2: Model H attention before and after reasoning finetuning, early layer 0 (upper row of each block) and late layer 9 (lower row), three heads each. Query position runs down, key position across. Everything above the diagonal is exactly zero in every panel, which is the causal mask made visible. In the pretrained model layer 9 shows vertical stripes where many queries converge on a few informative key positions; after finetuning those late-layer panels pull in toward the diagonal.

5 Phase 3: reasoning finetuning

The finetuning data is generated programmatically by `common/reasoning/generate.py`, entirely in each model's own language and script. The generator draws the values first and derives the answer from them, so labels are correct by construction and nothing is annotated. There are three task types: `compare2` gives two entities an attribute value each and asks which is greater or smaller (chance 1/2); `order3` does the same with three (chance 1/3); and `transitive` supplies $A<B$ and $B<C$ with no numbers at all and asks for a comparison that requires chaining (chance 1/2). Attributes are age, height, weight and price, each carrying its own grammatical agreement forms so that prompts read naturally rather than as filled slots. Each language gets 45,000 examples: 40,000 train, 2,000 validation, 2,000 test and 1,000 in a separate unseen-entity probe.

I picked 40,000 rather than deriving it, so afterwards I measured what it bought: finetuning at 5k, 10k, 20k and 40k with everything else held fixed, each scored on the same held-out test split. Test exact match rises 0.4660, 0.5950, 0.7875, 0.8460 for Hindi and 0.4440, 0.4460, 0.6000, 0.7555 for Assamese.

Assamese needs roughly twice the data to reach the same place. Hindi leaves its chance baseline between 5k and 10k; Assamese is still sitting on chance at 10k. At 40k Hindi is decelerating (0.0585 over the last doubling) while Assamese still gains 0.1555. The model that saw 18.4% less language in pretraining is also the one furthest from its ceiling here, which is the same resource story at a second, independent point.

Both curves are still rising at 40,000, so that figure was the ceiling imposed by the entity pools rather than a demonstrated sufficiency. More data would likely have helped both. Per-task lines from these single runs are noisy and I would not read the wiggles in them.

The spec allows either held-out entity names or held-out relation patterns as a leakage control. I used both, as two separate test splits, because they turn out to measure quite different things. The `test` split uses three templates that never appear in training while keeping the same entity pool, so it tests generalisation over phrasing. The `test_unseen` split keeps the training templates but substitutes 14 names and objects that never occur in training, testing generalisation over entities. I verified both properties on the shipped JSONL files rather than assuming them: 0 of 3 test templates appear in training, and 0 of 14 unseen entities do.

Each model finetunes from its own pretrained checkpoint with its own tokenizer held fixed, and neither sees the other's data. Three epochs, 3,750 steps, batch 32, learning rate 5×10^{-5} decaying by cosine to 5×10^{-6} with 5% warmup, seed 3407, 2.9 minutes each.

Three implementation decisions mattered far more than any hyperparameter, and I reached each only after measuring a failure.

1. **Loss on the answer span alone**, with prompt positions set to the ignore index.
2. **EOS inside the supervised span.** Without it, exact match was 0.0005 against a prefix match of 0.5710: the model named the right entity and then never stopped.
3. **A rationale before the answer**, the most important of the three. Training on the final entity alone left both models at chance, 0.4355 against a 0.4512 baseline. Supervising the decisive value first, so the target reads 22 साल, इसलिए दीपक ("22 years, therefore Deepak"), moved Hindi to 0.8460.

Only the entity after the marker is scored, so correct working followed by a wrong conclusion still counts as zero.

Two bugs in the generator were caught the same way. The transitive question direction originally came from the same variable as the premise direction, which made the answer entity position 0 in 100% of cases, so a model could score perfectly by echoing the first name it saw. Drawing the two independently fixed it. Separately, using disjoint entity pools for train and validation made the task unlearnable rather

than merely hard: validation loss climbed from 4.21 to 15.43 while training loss fell to 0.75, because none of the validation answer tokens had ever been seen.

Table 5: Reasoning accuracy under free generation (prompt only, greedy decoding to EOS, exact string match) on the held-out test split, n=2,000 per language. The error analysis classifies every prediction by whether the value and the owner were correct.

Accuracy	Model H	Model L	chance	Error structure	Model H	Model L
Pretrained (overall)	0.0000	0.0000	0.4512/0.4488	binding failure	0.1290	0.2245
Finetuned (overall)	**0.8460**	**0.7555**	0.4512/0.4488	comparison failure	0.0250	0.0200
transitive	**1.0000**	**1.0000**	0.5000	binding % of errors	83.8%	91.8%
compare2	0.9600	0.8928	0.5000	**value selected correctly**	**0.9750**	**0.9800**
order3	0.5282	**0.3436**	**0.3333**	P(owner | value), order3	0.5479	**0.3576**
`test_unseen`	0.0120	0.0000	0.4492/0.4510	P(owner | value), compare2	0.9961	0.9109

Pretrained accuracy is exactly zero for both models, which is not a scoring artefact. Prefix match is nonzero (0.2100 and 0.0715) but still below the chance baseline, so the base models genuinely have no notion of the task; finetuning installs an ability rather than sharpening one. At the other end, both models solve transitive chaining perfectly on 615 and 584 held-out items. That is the task with no numbers in it and the one closest to the multi-hop reasoning the brief asks about, and the resource tier makes no difference to it whatsoever.

The nine-point overall gap turns out to live almost entirely in one task. The models are identical on transitive and 6.7 points apart on `compare2`, but on `order3` Model L scores 0.3436 against a 0.3333 chance baseline, with Model H at 0.5282.

That reads like Model L never learned three-way ordering, and I checked, because it would be a strong claim. It is not what happened. Scoring the full *validation* split with the same free-generation method gives Model L **0.7719** on `order3` and Model H **0.9381**, both far above chance. The difference is that validation reuses the training templates while the test split holds its templates out, so what collapses is not the skill but its transfer to one unseen phrasing: a 43-point drop for Model L and 41 for Model H. Both models learned three-way ordering; Model L's version of it survives rephrasing much less well. This also means the per-task figures should be read with care, since the test split contains a single template per task and task difficulty is therefore confounded with which phrasing happened to be held out. The Model H versus Model L comparison is unaffected, as both were scored on identical prompts.

Both models then collapse on `test_unseen`, to 0.0120 and 0.0000, far below chance, while scoring 0.85 and 0.76 on held-out phrasings. They generalise over how a question is worded and fail completely over which entities it names, so the comparison operation is learned but welded to a memorised entity set.

The error analysis changed my reading of these numbers. Splitting each prediction into whether the right value was selected and whether it reached the right owner shows both models performing the comparison itself at 97.50% and 98.00%, with the lower-resource model marginally ahead: 40 comparison errors against Model H's 50.

Between 84% and 92% of all errors are a correct comparison attached to the wrong name. Conditioned on the comparison succeeding, Model L names the right owner 35.8% of the time on `order3` against a 33.3% baseline. It identifies the answer, then guesses whose it is.

Binding count is the natural explanation for the task ordering: none in transitive, one in `compare2`, two in `order3`. I would not push it further than the evidence goes, though. With one template per task, binding count and template are confounded, and on the training templates Model H scores *higher* on `order3` (0.9381) than `compare2` (0.8889), the reverse of the prediction.

Qualitative successes and failures, verbatim greedy generation on held-out test items. The rationale, marker and answer are all produced by the model.

Hit सुधीर की लंबाई ओमकार से कम पाई गई और ओमकार की लंबाई विजय से कम पाई गई। इस आधार पर सुधीर और विजय में कम लंबाई किसकी है? → क्रम सुधीर, ओमकार, विजय, इसलिए सुधीर, gold सुधीर. The premises run one way and the question the other, so the model writes out the chain it inferred and then picks the correct end rather than echoing the first entity.

Hit জগদীশ, উদয় আৰু মাধৱৰ বয়স ক্ৰমে 12 বছৰ, 63 বছৰ আৰু 9 বছৰ। কাৰ বয়স আটাইতকৈ কম? → 9 বছৰ, গতিকে মাধৱ, gold মাধৱ.

Miss सतीश, आशीष और नरेश की उम्र क्रमशः 76 साल, 62 साल और 60 साल है। सबसे ज़्यादा उम्र किसकी है? → 76 साल, इसलिए आशीष, gold सतीश. 76 is the correct maximum, so the comparison worked and the attribution did not.

Miss হেমন্ত, অনিল আৰু ৰামৰ বয়স ক্ৰমে 47 বছৰ, 37 বছৰ আৰু 22 বছৰ। কাৰ বয়স আটাইতকৈ বেছি? → 47 বছৰ, গতিকে মুকেশ, gold হেমন্ত. 47 is also correct, but মুকেশ does not appear in the prompt at all and has been recalled from the training pool.

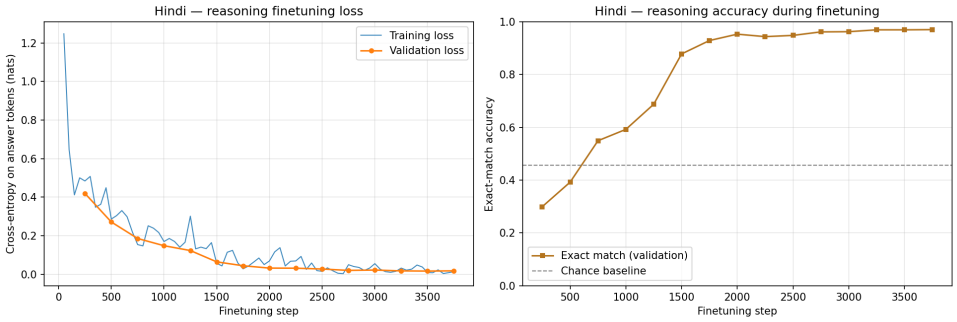


Figure 3: Model H finetuning: loss on the answer span (left) and validation exact match (right), which crosses the chance line near step 600 and plateaus by step 2,000. Model L follows the same shape at a lower ceiling.

6 The four questions

6.1 How did data scale and quality differ?

Scale barely differed, by construction. Hindi has 589.95M training tokens and Assamese 591.26M, a 0.2% difference with Assamese marginally ahead, and both models then trained on the identical 589,824,000-token schedule. That parity is the load-bearing control of the project. Reaching it for Assamese took six expansion rounds.

Token parity is not exposure parity, though. Assamese's 591M tokens represent only 373.9M words against Hindi's 453.5M, because its tokenizer needs 22.6% more tokens per word.

Quality differed in composition rather than cleanliness. Both corpora went through the same pipeline, and Assamese ended with the higher manual share (23.88% against 20.80%). But Hindi's downloaded tier draws on large, editorially varied web and news corpora while Assamese's is narrower and leans on OCR'd books. Held-out unknown rate (0.186% against 0.119%) and byte-fallback rate (0.650% against 0.430%) are both around 50% higher for Assamese, which is a measurable trace of that.

6.2 How do language-modelling and reasoning results compare across the tiers?

Language modelling shows a consistent but modest gap: 11% more bits-per-byte. Perplexity's apparent 95% is an artefact of comparing per-token quantities across different tokenizers.

Reasoning is less a gap than a cliff at one difficulty. The models are identical on transitive, 6.7 points apart on `compare2`, and Model L sits at chance on `order3`. Both tiers learn symbolic multi-hop chaining perfectly. The lower-resource model fails only where several value-entity bindings must be held at once, and there it fails outright rather than degrading gracefully. The resource tier does not scale reasoning ability smoothly so much as set a ceiling on task complexity.

One limit the tier does not explain: both models score near zero on `test_unseen`, so entity-level abstraction is a capacity limit of 24M parameters and 40k examples rather than anything to do with Assamese.

6.3 What tokenizer or corpus factors most affected the lower-resource model?

Fertility, by a wide margin: 1.6019 against 1.3067 at an identical 16k vocabulary. Three consequences follow, all measured.

- At the matched token budget, 589,824,000/1.6019 ≈ 368.2M words against Hindi's 451.4M, so **Model L saw 18.4% less language for the same compute**.
- It makes more predictions per unit of text, which is the mechanism behind the inflated perplexity.
- 34.2% of Assamese reasoning answers are multi-token against 17.2% of Hindi's, exactly twice as often, so exact match demands more consecutive correct predictions from the weaker model.

Fertility is higher because Assamese is more agglutinative. Bare case-suffix pieces such as ৰ, ত and ক rank in its top 15 tokens, whereas Hindi's top tokens are whole postpositions like के, है and में. BPE has to spend Assamese vocabulary on morpheme fragments that Hindi can spend on whole words.

One structural constraint is worth recording. The 20% manual floor caps total corpus size at `manual/0.20`, a ceiling only manual collection can lift, and at one point that ceiling sat at roughly 489M tokens, below target before subtracting held-out data.

6.4 What evidence explains the observed differences?

The chain is short and every link is measured.

1. Token budgets matched to 0.2% eliminate quantity as an explanation.
2. Fertility 22.6% higher converts that matched budget into 18.4% less language seen. This is arithmetic, not conjecture.
3. An 11% bits-per-byte penalty from 18% less effective exposure, at identical capacity, is a proportionate outcome.
4. The reasoning results localise the deficit to tasks requiring multiple bindings.
5. The error analysis then names the mechanism, and it is not the one the aggregate scores suggest.

That fifth step is the one that matters. Both models compare values at 97.50% and 98.00%, so the gap is entirely about binding. Binding a number to a name across intervening text is a language-modelling competence rather than an arithmetic one: exactly what less exposure and worse bits-per-byte would degrade, and exactly what the comparison operation does not need.

Two further checks agree. The attention analysis shows the weaker model perturbed further by finetuning (−3.20 against −1.76 tokens, 42 heads against 38). The shared `test_unseen` collapse isolates what is *not* a resource effect at all.

Put together: equal token budgets plus unequal fertility produced unequal linguistic exposure, which produced a proportionate language-modelling gap, which surfaced in reasoning only where the task demanded several simultaneous value-entity bindings.

7 Reproduction and optional bonus

Code lives in `common/`: `modeling/` (model, training, evaluation, attention, verification), `reasoning/` (generation, finetuning, scoring, error analysis) and `clean/`, `corpus/`, `sources/`, `tokenization/` for Phase 1. Per-language artifacts sit under `<lang>/`, figures in `report/figures/`, notebooks in `kaggle/`.

Each step runs as `python -m common.<module>` with `--lang hindi` or `--lang assamese`, in this order: `tokenize_corpus`; then `train` (which resumes from `last.pt`), `evaluate` and `attention` from `modeling`; then `generate`, `finetune`, `evaluate`, `error_analysis` and `attention_compare` from `reasoning`. The 40k training split and the tokenized arrays regenerate deterministically rather than being committed; validation, test and `test_unseen` are committed, so every number here reproduces exactly.

I also completed the optional ablation, on branch `bonus-ablation` with its own write-up in `report/bonus_report.pdf`. Hindi was retrained with the positional table removed rather than zeroed, everything else identical, costing 196,608 parameters. Bits-per-byte moved only from 0.5595 to 0.5656, far less than the usual argument for positional embeddings predicts.

A linear probe explains why. Position stays recoverable from the final hidden state at $R^2 = 0.8121$ against the standard model's 0.8183, so the ablated model keeps 99.2% of its positional decodability with no positional parameters at all. Causal masking already encodes position: a query at index t attends to exactly $t{+}1$ keys, and that count measures where it sits. What degrades is repetition (0.072 to 0.118 at $\tau{=}1.0$) and layer 0, whose entropy drops 0.937 to 0.661 as it takes over establishing position.